UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO - UFERSA
CENTRO MULTIDISCIPLINAR DE PAU DOS FERROS – CMPF
DEPARTAMENTO DE ENGENHARIA E TECNOLOGIA
DISCIPLINA: PROGRAMAÇÃO CONCORRENTE E DISTRIBUÍDA

QUESTÕES RESPONDIDAS UNIDADE I

FERNANDO UMBILINO ALVES

PAU DOS FERROS – RN
2025
QUESTÕES RESPONDIDAS

- QUESTÃO 01
- QUESTÃO 02
- QUESTÃO 03
- QUESTÃO 04
- QUESTÃO 05
- QUESTÃO 06
- QUESTÃO 07
- QUESTÃO 08
- QUESTÃO 09
- QUESTÃO 10
- QUESTÃO 11
- QUESTÃO 12
- QUESTÃO 13
- QUESTÃO 14
- QUESTÃO 15
- QUESTÃO 16
- QUESTÃO 17
- QUESTÃO 18
- QUESTÃO 19

PORQUE COMPUTAÇÃO PARALELA?

QUESTÃO 01

Suponha que precisamos calcular *n* valores e somá-los. Suponha que também tenhamos *p* núcleos e *p* seja muito menor que *n*. Então cada núcleo pode calcular uma soma parcial de aproximadamente valores *n/p* da seguinte maneira:

```
minha_soma = 0;
meu_pri_i = . . . ;
meu_ult_i = . . . ;
for (meu_i = meu_pri_i; meu_i < meu_ult_i; meu_i+=meu_desl) {
      meu_x = Compute_prox_valor(. . .);
      minha_soma += meu_x;
}
```

Aqui o prefixo *meu_* indica que cada núcleo está usando suas próprias variáveis privadas e cada núcleo pode executar este bloco de código independentemente dos outros núcleos.

Supondo que cada chamada para *Compute_prox_valor* requer aproximadamente a mesma quantidade de trabalho, elabore fórmulas para calcular *meu_pri_i*, *meu_ult_i* e *meu_desl*. Lembre-se de que cada núcleo deve receber aproximadamente o mesmo número de elementos de computação no loop.

Dica: primeiro considere o caso em que *n* é divisível por *p*. A partir daí, elabore fórmulas para o caso em que *n* não é divisível por *p*.

Divisão de trabalho por bloco entre os núcleos:

```C/C++
n = quantidade de valores;
p = quantidade de núcleos;
resto = n % p;
inteiro = parte inteira de n/p;
meu_desl = 1;

//zona paralela
my_rank = índice do núcleo;
minha_soma = 0;
if(resto == 0){
      meu_pri_i = my_rank*(inteiro);
      meu_ult_i = (my_rank+1)*(inteiro);
}else{
      if(my_rank < resto){
            meu_pri_i = my_rank*(inteiro) + my_rank;
            meu_ult_i = (my_rank + 1)*(inteiro) + my_rank + 1;
      }else{
            meu_pri_i = my_rank*(inteiro) + resto;
            meu_ult_i = (my_rank+1)*(inteiro) + resto;
      }
for (meu_i = meu_pri_i; meu_i < meu_ult_i; meu_i+=meu_desl) {
```

```
meu_x = Compute_prox_valor(. . .);
minha_soma += meu_x;
}
```

QUESTÃO 02

Suponha que precisamos calcular *n* valores e somá-los. Suponha que também tenhamos *p* núcleos e *p* seja muito menor que *n*. Então cada núcleo pode calcular uma soma parcial de aproximadamente valores *n/p* da seguinte maneira:

```
minha_soma = 0;
meu_pri_i = . . . ;
meu_ult_i = . . . ;
for (meu_i = meu_pri_i; meu_i < meu_ult_i; meu_i+=meu_desl) {
      meu_x = Compute_prox_valor(. . .);
      minha_soma += meu_x;
}
```

Aqui o prefixo *meu_* indica que cada núcleo está usando suas próprias variáveis privadas e cada núcleo pode executar este bloco de código independentemente dos outros núcleos.

Considerando que a chamada com $i = k$ requer $k + 1$ vezes mais trabalho que a chamada com $i = 0$ (se a chamada com $i = 0$ requer *2* milissegundos, a chamada com $i = 1$ requer *4*, a chamada com $i = 2$ requer *6*...), elabore fórmulas para calcular *meu_pri_i*, *meu_ult_i* e *meu_desl*. Comparado com a distribuição em blocos contíguos de iterações, sua fórmula deve reduzir a diferença do tempo de execução entre os núcleos.

Divisão de trabalho de forma cíclica entre os núcleos:

```C/C++
n = quantidade de valores;
p = quantidade de núcleos;

//zona paralela
my_rank = índice do núcleo;
minha_soma = 0;
meu_pri_i = my_rank;
meu_ult_i = n;
meu_desl = p;
for (meu_i = meu_pri_i; meu_i < meu_ult_i; meu_i+=meu_desl) {
      meu_x = Compute_prox_valor(. . .);
      minha_soma += meu_x;
}
```

Comparando o pseudocódigo acima com o da Questão 1 e usando como exemplo $n = 12$ valores para serem somados e $p = 4$ núcleos, podemos observar na comparação entre as tabelas abaixo, a diferença da divisão de trabalho entre os núcleos em ambos os cenários. Na situação descrita na questão anterior, o núcleo 0 vai executar as posições 0, 1 e 2, o núcleo 1 vai executar as posições 4, 5 e 6, e assim por diante, dessa forma, o trabalho dos núcleos, na situação descrita nessa questão, vai ficar extremamente desbalanceado.

Representação da situação aplicada na Questão 1:

NÚCLEOS	ÍNDICES			TEMPO
0	0	1	2	*12ms*
1	3	4	5	*30ms*
2	6	7	8	*48ms*
3	9	10	11	*66ms*

O pseudocódigo da Questão 2 busca reduzir essa diferença de tempo entre os núcleos, realizando uma distribuição do trabalho de forma mais eficiente, como exemplificado abaixo:

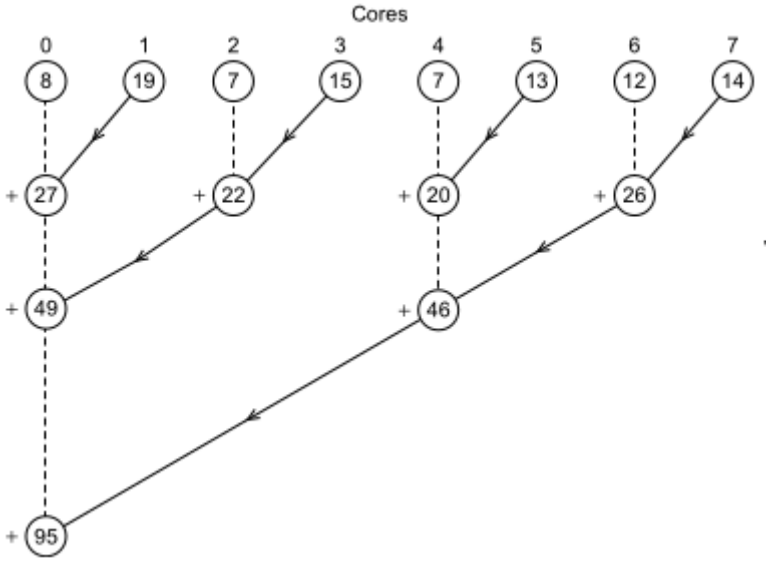
NÚCLEOS	ÍNDICES			TEMPO
0	0	4	8	*30ms*
1	1	5	9	*36ms*
2	2	6	10	*42ms*
3	3	7	11	*48ms*

Analisando as tabelas, podemos observar que a diferença do tempo de execução entre os núcleos reduziu significativamente com o segundo pseudocódigo.

QUESTÃO 03

Escreva um pseudocódigo para uma soma global estruturada em árvore. Suponha que o número de núcleos seja uma potência de dois.

O objetivo de uma soma global estruturada em árvore é pegar um valor numérico de cada thread que está participando de um programa paralelo e a cada fase com um parceiro calcular a soma e no final, uma única thread deve ter o resultado total.




```
C/C++
p = quantidade de núcleos;
my_rank = índice do núcleo;
rank_parceiro = índice da dupla;
expoente = 2;
distancia = 1;

while (distancia < p){
	if(my_rank % distancia == 0){
		if(my_rank % expoente == 0){
			rank_parceiro = my_rank + distancia;
			//Recebo o valor da dupla e somo

		}else{
			rank_parceiro = my_rank - distancia;
			//Envio meu valor para a dupla
			break;
		}
	}
	expoente *= 2;
	distancia *= 2;
}
```

QUESTÃO 04

Podemos usar os operadores bit a bit de C para implementar uma soma global estruturada em árvore. Implemente este algoritmo em pseudocódigo usando o operador OU EXCLUSIVO e o operador *shift* para esquerda.

O objetivo de uma soma global estruturada em árvore é pegar um valor numérico de cada thread que está participando de um programa paralelo e a cada fase com um parceiro calcular a soma e no final, uma única thread deve ter o resultado total.

Na tabela abaixo, é possível observar que ao aplicar o operador OU EXCLUSIVO entre o deslocamento e o índice da thread, é possível encontrar o parceiro.

ÍNDICE DA THREAD	**DESLOCAMENTO**		
	$\mathbf{1_{10} = 0001_2}$	$\mathbf{2_{10} = 0010_2}$	$\mathbf{4_{10} = 0100_2}$
$\mathbf{0_{10} = 0000_2}$	$1_{10} = 0001_2$	$2_{10} = 0010_2$	$4_{10} = 0100_2$
$\mathbf{1_{10} = 0001_2}$	$0_{10} = 0000_2$	–	–
$\mathbf{2_{10} = 0010_2}$	$3_{10} = 0011_2$	$0_{10} = 0000_2$	–
$\mathbf{3_{10} = 0011_2}$	$2_{10} = 0010_2$	–	–
$\mathbf{4_{10} = 0100_2}$	$5_{10} = 0101_2$	$6_{10} = 0110_2$	$0_{10} = 0000_2$
$\mathbf{5_{10} = 0101_2}$	$4_{10} = 0100_2$	–	–
$\mathbf{6_{10} = 0110_2}$	$7_{10} = 0111_2$	$4_{10} = 0100_2$	–
$\mathbf{7_{10} = 0111_2}$	$6_{10} = 0110_2$	–	–

No pseudocódigo, faz o uso do OU EXCLUSIVO para encontrar o parceiro e uso do operador *shift* para esquerda encontrando distancia.

```
C/C++
p = quantidade de núcleos;
my_rank = índice do núcleo;
rank_parceiro = índice da dupla;
distancia = 1;

while (distancia < p){
       if(my_rank % distancia == 0){
              rank_parceiro = my_rank ^ distancia;
              if(my_rank < rank_parceiro){
                     //Recebo o valor da dupla e somo

              }else{
                     //Envio meu valor para a dupla
                     break;
```

```
            }
        }
        distancia = distancia << 1;
}
```

QUESTÃO 05

Escreva um pseudocódigo para uma soma global estruturada em árvore. Considere que o número de núcleos pode não ser uma potência de dois.

O objetivo de uma soma global estruturada em árvore é pegar um valor numérico de cada thread que está participando de um programa paralelo e a cada fase com um parceiro calcular a soma e no final, uma única thread deve ter o resultado total.

```C/C++
p = quantidade de núcleos;
my_rank = índice do núcleo;
rank_parceiro = índice da dupla;
expoente = 2;
distancia = 1;

while (distancia < p){
	if(my_rank % distancia == 0){
		if(my_rank % expoente == 0){
			rank_parceiro = my_rank + distancia;
			if (rank_parceiro < p) {
			//Recebo o valor da dupla e somo
			}
		}else{
			rank_parceiro = my_rank - distancia;
			//Envio meu valor para a dupla
			break;
		}
	}
	expoente *= 2;
	distancia *= 2;
}
```

QUESTÃO 06

Elabore fórmulas para o número de recebimentos e adições que o núcleo 0 realiza usando

(a) uma soma global onde apenas o núcleo 0 realiza as adições;

Nesse cenário, onde todos os núcleos enviam para o núcleo 0, temos que:

$$quant_total = n - 1$$

Onde *n* é o número de núcleos e *quant_total* o número de recebimentos e adições.

(b) uma soma global estruturada em árvore.

No cenário da soma global estruturada em árvore, temos a equação:

$$quant_total = log_2(n)$$

Onde n é o número de núcleos e quant_total o número de recebimentos e adições.

Faça uma tabela mostrando os números de recebimentos e adições realizados pelo núcleo 0 quando as duas somas são usadas com 2, 4, 8, · · · , 1024 núcleos.

	Quantidade recebimentos e adições	
Núcleos (*n*)	**n - 1**	***$log_2(n)$***
2	1	1
4	3	2
8	7	3
16	15	4
32	31	5
64	63	6
128	127	7
256	255	8
512	511	9
1024	1023	10

QUESTÃO 07

Descreva um problema de pesquisa em sua área (Tecnologia de Informação, Engenharia de Software...) que se beneficiaria com o uso da computação paralela. Forneça um esboço de como o paralelismo seria usado. Você usaria paralelismo de tarefas ou de dados? Por quê?

Na área de Tecnologia da Informação (TI), o crescimento exponencial no volume de dados gerados diariamente impõe um desafio significativo de gerenciamento e processamento, conhecido como Big Data. Essa imensa quantidade de informação é crucial para impulsionar o desenvolvimento em diversos setores, como pesquisa científica, saúde e economia. No entanto, o aumento da escala de dados leva a uma queda no desempenho das máquinas utilizadas para análise, resultando em processos mais lentos e atrasos em atividades críticas, como a descoberta de tratamentos para doenças. Diante desse cenário, a adoção de estratégias para reduzir o tempo de análise desses dados e obter resultados valiosos de forma eficiente torna-se fundamental.

Uma solução viável para esse problema é a computação paralela, que distribui o processamento dos dados entre múltiplos processadores, permitindo que diferentes partes da análise sejam realizadas simultaneamente, acelerando assim a obtenção de resultados. No contexto de Big Data, o paralelismo seria empregado em diversas fases, desde o momento onde múltiplos núcleos leriam e filtrariam dados de diferentes fontes, passando pela extração de características (calculando atributos em diferentes blocos de dados em paralelo), até a aplicação de algoritmos de aprendizado de máquina (distribuindo o treinamento ou a inferência por diversas unidades de processamento) e a geração de alerta.

Considerando a natureza do problema, que envolve o processamento de uma quantidade massiva de dados, a abordagem adequada seria uma combinação de paralelismo de dados e paralelismo de tarefas. O paralelismo de dados divide o excesso de dados em partes menores e as processa independentemente em paralelo, maximizando a eficiência para lidar com o volume. Por exemplo, diferentes máquinas ou núcleos podem analisar diferentes lotes de dados de rede para detecção de anomalias. Já o paralelismo de tarefas seria utilizado para dividir as diferentes etapas da análise (leitura de dados, processamento, análise, alerta), permitindo que estágios distintos operem em paralelo em diferentes conjuntos de dados. Essa abordagem híbrida otimiza a utilização dos recursos, garantindo a agilidade necessária para atender às demandas de setores críticos e promover avanços.

HARDWARE E SOFTWARE PARALELOS

QUESTÃO 08

Quando discutimos a adição de ponto flutuante, partimos da suposição simplificadora de que cada uma das unidades funcionais levava o mesmo tempo. Suponha que buscar (*fetch*) e armazenar (*store*) levem 2 nanossegundos cada e que as operações restantes levem 1 nanossegundo cada.

(a) Quanto tempo leva uma adição de ponto flutuante com essas suposições?

Tempo	**Operação**
2	Buscar operandos (*fetch*)
1	Comparar expoentes
1	Deslocar um operando
1	Somar
1	Normalizar resultado
1	Arredondar resultado
2	Armazenar resultado (store)
9 ns	**Tempo total**

(b) Quanto tempo levará uma adição sem pipeline de 1.000 pares de floats com essas suposições?

Sem pipeline (ou sequencial), uma operação completa deve terminar todas as suas etapas antes que a próxima operação possa começar, sabendo que o tempo total em uma adição é *9 ns*, então:

$$9\ ns * 1000 = 9000\ ns$$

(c) Quanto tempo levará uma adição em pipeline de 1.000 pares de floats com essas suposições?

Com pipeline, as etapas das operações se sobrepõem no tempo. A primeira adição leva o tempo total de *9 ns* para preencher o pipeline, mas às 999 adições subsequentes são concluídas no ritmo da etapa mais longa (*2 ns*), então:

$$9\ ns + (1000 - 1) * 2\ ns = 2007\ ns$$

Onde, *9 ns* é o tempo da primeira operação que percorre todo o pipeline, *1000 - 1* é as operações restantes e *2 ns* é o tempo da etapa mais longa.

(d) O tempo necessário para busca e armazenamento pode variar consideravelmente se os operandos/resultados forem armazenados em diferentes níveis da hierarquia de memória. Suponha que uma busca de um cache de nível 1 leve dois nanossegundos,

enquanto uma busca de um cache de nível 2 leve cinco nanossegundos e uma busca da memória principal leve cinquenta nanossegundos. O que acontece com o pipeline quando há uma falha de cache de nível 1 na busca de um dos operandos? O que acontece quando há uma falha de nível 2?

Quando ocorre uma falha de cache de nível 1 durante a busca de um operando, o processador é obrigado a buscar no cache de nível 2. Então, se a busca no cache L1 levaria normalmente *2ns*, ao falhar e precisar acessar o cache L2 (*5ns*), o tempo total para obter o dado passa a ser de *7ns*.

Se houver uma falha no cache de nível 2, é necessário buscar o dado diretamente na memória principal, cujo tempo de acesso é mais elevado. Nesse cenário, o tempo total para obter o dado será de 57ns.

QUESTÃO 09

Explique como uma fila implementada em hardware na CPU poderia ser usada para melhorar o desempenho de um cache *write-through*.

Em um cache com *write-through*, onde cada escrita é imediatamente enviada para a memória principal, a CPU normalmente precisaria parar para fazer a escrita do dado na memória, gerando pausas. Para contornar essa lentidão, uma fila implementada em hardware, atua como uma fila de escrita posicionada entre a CPU e a memória principal, ao invés de atualizar a memória principal, a CPU envia os dados para a fila de escrita, permitindo que a CPU prossiga com a execução de outras instruções imediatamente. Enquanto a CPU continua seu trabalho, a fila de escrita assume a responsabilidade de gravar os dados na memória principal em segundo plano, de forma assíncrona.

QUESTÃO 10

Dado o exemplo a seguir envolvendo leituras de cache de um array bidimensional.

```
double A[MAX][MAX], x[MAX], y[MAX];
. . .
// Initializa A e x; y = 0
. . .
// Primeiro par de loops/
for (i = 0; i < MAX ; i++)
       for (j = 0; j < MAX; j++)
              y [i] += A[i][j]x[j];
. . .
// y = 0
. . .
// Segundo par de loops
for (j = 0; j < MAX ; j++)
       for (i = 0; i < MAX; i++)
              y [i] += A[i][j]x[j];
```

(a) Como o aumento do tamanho da matriz afetaria o desempenho dos dois pares de loops aninhados?

Quando aumentamos o tamanho da matriz, o desempenho dos dois pares de loops aninhados tende a piorar devido ao maior volume de dados, ao aumento dos acessos à memória e das falhas de cache. No primeiro par de loops, o acesso em A ocorre linha a linha, aproveitando bem a localidade espacial e mantendo um desempenho razoável. Já no segundo par de loops, o acesso em A se dá coluna a coluna, forçando saltos na memória para cada elemento, essa falta de localidade espacial faz com que o número de falhas de cache aumente dramaticamente à medida que a matriz cresce, resultando em uma degradação de desempenho muito maior.

(b) Como o aumento do tamanho da cache afetaria o desempenho dos dois pares de loops aninhados?

Ao aumentar o tamanho da cache, o desempenho melhora para ambos os loops, mas o benefício é distribuído de forma desigual. O primeiro par de loops, que já possui excelente localidade espacial ao acessar a matriz A linha a linha, terá um ganho de desempenho muito mais significativo. Uma cache maior permitiria que mais linhas completas da matriz fossem mantidas no cache, reduzindo drasticamente as falhas e acelerando o processamento. O segundo par de loops, por sua vez, também se beneficiaria, mas de forma menos expressiva. Embora mais linhas dispersas da matriz pudessem ser armazenadas, o modo de acesso a elementos não contíguos (saltando entre linhas na memória para cada elemento de uma coluna) persiste, o que limita a eficácia do cache e mantém seu desempenho muito inferior ao do primeiro loop.

(c) Quantas falhas ocorrem nas leituras de A no primeiro par de loops aninhados?

Como cada linha da matriz tem *MAX* elementos, ela requer *MAX / 4* falhas de cache por linha para ser totalmente acessada, considerando que existem *MAX* linhas na matriz, o número total de falhas de cache será *MAX* (linhas) * (*MAX / 4*) (falhas por linha), resultando em $\boldsymbol{MAX^2 / 4}$ falhas de cache.

(d) Quantas falhas ocorrem no segundo par?

Como há *MAX* acessos por coluna (i de 0 até MAX - 1) e *MAX* colunas no total (j de 0 até MAX - 1), o número total de falhas de cache para a matriz A será *MAX* (colunas) * *MAX* (acessos por coluna), resultando em $\boldsymbol{MAX^2}$ falhas de cache.

Suponha que uma linha de cache consiga armazenar quatro elementos de A.

QUESTÃO 11

Responda as questões abaixo. Explique o por quê de cada resposta.

(a) A adição de cache e memória virtual a um sistema von Neumann altera sua designação como sistema SISD?

Não. Nem a cache, nem a memória virtual alteram o modelo fundamental de processamento do sistema. A CPU continua buscando e executando uma única instrução por vez e operando sobre um único item de dado por vez. Essas são otimizações da hierarquia de memória e do gerenciamento de memória, respectivamente, mas não introduzem múltiplos fluxos de instruções independentes ou múltiplos fluxos de dados processados simultaneamente em paralelo.

(b) E quanto à adição de pipeline?

Não, embora várias instruções estejam ativas no pipeline ao mesmo tempo, elas pertencem a uma única sequência lógica de instruções de um único programa (ou thread). O processador continua seguindo um único contador de programa e operando sobre um único fluxo de dados como ditado por essa sequência. Ele não é capaz de executar múltiplos programas ou threads independentes.

(c) Multiple issue?

Não, a técnica de Multiple Issue se refere à capacidade de um processador de iniciar a execução de múltiplas instruções em um único ciclo de clock. Embora um processador superescalar possa iniciar várias instruções no mesmo ciclo de clock, essas instruções ainda fazem parte de um único fluxo de instruções, ou seja, da mesma thread ou programa, e operam sobre o fluxo de dados dessa mesma thread.

(d) Hardware multithreading?

Sim, com o multithreading por hardware, o sistema passa a ter a capacidade de processar múltiplos fluxos de instruções independentes no mesmo núcleo, consequentemente, operam sobre múltiplos fluxos de dados independentes. Isso aproxima das características de um sistema MIMD ao nível de núcleo, pois um único núcleo pode gerenciar e executar efetivamente múltiplas threads independentes e seus respectivos dados.

QUESTÃO 12

Suponha que um programa deva executar 10^{12} instruções para resolver um problema específico. Suponha ainda que um sistema de processador único possa resolver o problema em 10^6 segundos (cerca de 11, 6 dias). Portanto, em média, o sistema de processador único executa 10^6 instruções por segundo. Agora suponha que o programa tenha sido paralelizado para execução em um sistema de memória distribuída. Suponha também que o programa paralelo usa p processadores e, assim, cada processador executará $10^{12}/p$ instruções. Além disso, cada processador deverá enviar $10^9 (p - 1)$ mensagens. Por fim, suponha que não haja *overhead* adicional na execução do programa paralelo. Ou seja, o programa será concluído depois que cada processador tiver executado todas as suas instruções e enviado todas as suas mensagens e não haverá atrasos devido a coisas como espera por mensagens.

(a) Suponha que demore 10^{-9} segundos para enviar uma mensagem. Quanto tempo levará para o programa ser executado com 1000 processadores se cada processador for tão rápido quanto o processador único no qual o programa serial foi executado?

Com a quantidade de 1000 processadores ($p = 1000$), cada processador vai executar:

$$10^{12}/10^3 = \mathbf{10^9}\ \textbf{instruções}$$

Cada processador também deverá enviar:

$$10^9 (1000 - 1) = \mathbf{10^9 * 999}\ \textbf{mensagens}$$

Logo, o tempo total para realizar todas as instruções, pode ser obtido da seguinte maneira:

1 segundo — 10^6 instruções

x segundos — 10^9 instruções

$$x * 10^6 = 10^9 * 1$$
$$x = 10^9/10^6$$
$$\mathbf{x = 10^3}\ \textbf{segundos}$$

E o tempo total para o envio das mensagens, pode ser obtido da seguinte forma:

$$999 * 10^9 * 10^{-9} = \mathbf{999}\ \textbf{segundos}$$

Com isso, a soma dos dois tempos obtemos o tempo total, supondo que na haja overhead adicional:

$$999 + 10^3 = \mathbf{1999}\ \textbf{segundos}$$

(b) Suponha que demore 10^{-3} segundos para enviar uma mensagem. Quanto tempo levará para o programa rodar com 1000 processadores?

Com a quantidade de 1000 processadores ($p = 1000$), cada processador vai executar:

$$10^{12}/10^3 = \mathbf{10^9}\ \textbf{instruções}$$

Cada processador também deverá enviar:

$$10^9 (1000 - 1) = \mathbf{10^9 * 999} \text{ mensagens}$$

Logo, o tempo total para realizar todas as instruções, pode ser obtido da seguinte maneira:

1 segundo	—	10^6 instruções
x segundos	—	10^9 instruções

$$x * 10^6 = 10^9 * 1$$
$$x = 10^9/10^6$$
$$\mathbf{x = 10^3} \text{ segundos}$$

E o tempo total para o envio das mensagens, pode ser obtido da seguinte forma:

$$999 * 10^9 * 10^{-3} = \mathbf{999 * 10^6} \text{ segundos}$$

Com isso, a soma dos dois tempos obtemos o tempo total, supondo que na haja overhead adicional:

$$= 999 * 10^6 + 10^3$$
$$= 999 * 1000000 + 1000$$
$$= 999000000 + 1000$$
$$= \mathbf{999001000} \text{ segundos}$$

QUESTÃO 13

(a) Suponha que um sistema de memória compartilhada use coerência de cache (*snooping*) e caches *write-back*. Suponha também que o núcleo *0* tenha a variável *x* em seu cache e execute a atribuição *x* = *5*. Finalmente, suponha que o núcleo *1* não tenha *x* em seu cache e, após a atualização do núcleo *0* para *x*, o núcleo *1* tente executar *y* = *x*. Qual valor será atribuído a *y*? Por que?

Em um sistema com coerência de cache por snooping e caches write-back, a variável *y* receberá o valor desatualizado de *x*. Quando o *núcleo 0* executa *x* = *5*, ele atualiza sua cópia local de *x* em seu cache e a marca como modificada, ao mesmo tempo, ele envia um sinal de invalidação para o barramento. O *núcleo 1* não tem *x* em seu cache inicialmente, quando ele for ler *x* vai pegar o valor desatualizado, pois o barramento só ia avisar para o *núcleo 0* atualizar se *núcleo 1* tivesse uma cópia de *x*.

(b) Suponha que o sistema de memória compartilhada da parte anterior utilize um protocolo baseado em diretório. Qual valor será atribuído a *y*? Por que?

Utilizando um protocolo baseado em diretório, a variável *y* também receberá o valor desatualizado. Quando o *núcleo 0* executa *x* = *5*, ele atualiza seu cache local e informa ao diretório sobre essa modificação. Já o diretório, que centraliza as informações de estado de todos os blocos de memória, registra o *núcleo 0* como o proprietário da cópia modificada de *x* e avisa a quem possui *x* que ele foi modificado, quando o *núcleo 1* tentar ler *x* vai pegar o valor desatualizado, pois ele não tinha um valor antigo de *x* então não foi avisado.

(c) Como os problemas encontrados nas duas primeiras partes podem ser resolvidos?

Utilizando o *Write-Through*, pois a consistência dos dados na memória principal seria garantida imediatamente após cada escrita.

QUESTÃO 14

Sendo n o tamanho do problema e p o número de threads (ou processos):

(a) Suponha que o tempo de execução de um programa sequencial seja dado por $T_{sequencial} = n^2$, onde as unidades do tempo de execução estão em microssegundos. Suponha que uma paralelização deste programa tenha tempo de execução $T_{paralelo} = n^2/p + log_2(p)$. Escreva um programa que encontre as acelerações e eficiências deste programa para vários valores de n e p. Execute seu programa com $n = 10, 20, 40, \cdots, 320$ e $p = 1, 2, 4, \cdots, 128$.

```
C/C++
#include <stdio.h>
#include <math.h>

double t_sequencial(int n) {
    return n * n;
}

double t_paralelo(int n, int p) {
    return (n * n) / (double)p + log2(p);
}

int main() {
    int n_valores[6] = {10, 20, 40, 80, 160, 320};
    int p_valores[8] = {1, 2, 4, 8, 16, 32, 64, 128};
    double sequencial_T, paralelo_T, speedups, eficiencia;

    printf("%-10s %-10s %-15s %-15s\n", "N", "P", "Speedups", "Eficiência");
    for (int i = 0; i < 6; i++) {
        int n = n_valores[i];
        sequencial_T = t_sequencial(n);

        for (int j = 0; j < 8; j++) {
            int p = p_valores[j];
            paralelo_T = t_paralelo(n, p);

            speedups = sequencial_T / paralelo_T;
            eficiencia = sequencial_T / (p * paralelo_T);

            printf("%-10d %-10d %-15.2f %-15.2f\n", n, p, speedups,
eficiencia);
        }

        printf("-----------------------------------------------\n");
    }

    return 0;}
```

SPEEDUPS

THREADS *(p)*	**TAMANHO DO PROBLEMA *(n)***					
	10	**20**	**40**	**80**	**160**	**320**
1	1.00	1.00	1.00	1.00	1.00	1.00
2	1.96	1.99	2.00	2.00	2.00	2.00
4	3.70	3.92	3.98	4.00	4.00	4.00
8	6.45	7.55	7.88	7.97	7.99	8.00
16	9.76	13.79	15.38	15.84	15.96	15.99
32	12.31	22.86	29.09	31.22	31.80	31.95
64	13.22	32.65	51.61	60.38	63.05	63.76
128	12.85	39.51	82.05	112.28	123.67	126.89

EFICIÊNCIA

THREADS *(p)*	**TAMANHO DO PROBLEMA *(n)***					
	10	**20**	**40**	**80**	**160**	**320**
1	1.00	1.00	1.00	1.00	1.00	1.00
2	0.98	1.00	1.00	1.00	1.00	1.00
4	0.93	0.98	1.00	1.00	1.00	1.00
8	0.81	0.94	0.99	1.00	1.00	1.00
16	0.61	0.86	0.96	0.99	1.00	1.00
32	0.38	0.71	0.91	0.98	0.99	1.00
64	0.21	0.51	0.81	0.94	0.99	1.00
128	0.10	0.31	0.64	0.88	0.97	0.99

- O que acontece com os *speedups* e eficiências à medida que *p* aumenta e *n* é mantido fixo?
  Quando *n* é fixo à medida que *p* aumenta, o *speedup* cresce de forma cada vez menos eficaz e a *eficiência* sempre diminui. Para problemas pequenos, o excesso de threads se mostra prejudicial, diminuindo o *speedup* e causando uma queda drástica na *eficiência*, isso indica que fatores como o *overhead* esteja prejudicando. Em contrapartida, para problemas grandes, a *eficiência* se mantém alta, e os ganhos de *speedup* continuam muito significativos, demonstrando que a paralelização é vantajosa.

- O que acontece quando p é fixo e n é aumentado?
  Quando o número de threads p é mantido fixo e o tamanho do problema n aumenta, tanto o *speedup* quanto a *eficiência* melhoram consistentemente, aproximando-se de seus valores ideais. Isso ocorre porque o custo de overhead, é relativamente fixo para um número constante de threads, enquanto o volume de trabalho computacional cresce significativamente com o aumento de n. Dessa forma, para problemas maiores, o tempo gasto em cálculos domina o tempo total de execução, "diluindo" o impacto do overhead. Como resultado, as threads são mais bem aproveitados, o que leva a *eficiência* a se aproximar de 100% e, consequentemente, o *speedup* a se aproximar do seu valor linear ideal.

(b) Suponha que $T_{paralelo} = T_{sequencial} / p + T_{overhead}$. Suponha também que mantemos p fixo e aumentamos o tamanho do problema.

- Mostre que, se $T_{overhead}$ crescer mais lentamente que $T_{sequencial}$, a eficiência paralela aumentará à medida que aumentarmos o tamanho do problema.
  Modificando o código da alternativa anterior, onde agora $T_{overhead}$ cresce mais lentamente que $T_{sequencial}$, temos que $T_{overhead}$ `= pow(n, 1.5)` e $T_{sequencial}$ `= pow(n, 2)`, com 64 threads.

```
C/C++
#include <stdio.h>
#include <math.h>

double t_sequencial(int n) {
    return pow(n, 2);
}

double t_overhead(int n) {
    return pow(n, 1.5);
}

int main() {
    int n_valores[6] = {10, 20, 40, 80, 160, 320};
    int p = 64;
    double sequencial_T, paralelo_T, speedups, eficiencia, overhead_T;

    printf("%-10s %-10s %-15s %-15s %-15s %-15s %-15s\n", "N", "P",
"Speedups", "Eficiência", "Tseque", "Tparal", "ToverH");
    for (int i = 0; i < 6; i++) {
        int n = n_valores[i];
        sequencial_T = t_sequencial(n);
        overhead_T = t_overhead(n);

        paralelo_T = (sequencial_T / p) + overhead_T;

        speedups = sequencial_T / paralelo_T;
        eficiencia = sequencial_T / (p * paralelo_T);
```

```
        printf("%-10d %-10d %-15.2f %-15.2f %-15.2f %-15.2f %-15.2f\n", n,
p, speedups, eficiencia, sequencial_T, paralelo_T, overhead_T);


printf("-----------------------------------------------------------------
---------------------------\n");
    }

    return 0;
}
```

A eficiência aumenta com o crescimento de *n*. Isso confirma que, quando o *overhead* cresce mais lentamente, o custo computacional domina o tempo total e o paralelismo se torna mais vantajoso.

EFICIÊNCIA

THREADS *(p)*	**TAMANHO DO PROBLEMA *(n)***					
	10	**20**	**40**	**80**	**160**	**320**
64	0.05	0.07	0.09	0.12	0.17	0.22

- Mostre que se, por outro lado, $T_{overhead}$ crescer mais rápido que $T_{sequencial}$, a eficiência paralela diminuirá à medida que aumentamos o tamanho do problema.
  Agora $T_{overhead}$ cresce mais rápido que $T_{sequencial}$, temos que $T_{overhead}$ `= pow(n, 2.5)` e $T_{sequencial}$ `= pow(n, 2)`, com 2 threads.

EFICIÊNCIA

THREADS *(p)*	**TAMANHO DO PROBLEMA *(n)***					
	10	**20**	**40**	**80**	**160**	**320**
64	0.14	0.10	0.07	0.05	0.04	0.03

A eficiência diminui com o crescimento de *n*. Como o *overhead* cresce mais rápido que o tempo de execução sequencial, ele passa a dominar o tempo total, reduzindo os benefícios do paralelismo.

QUESTÃO 15

Às vezes, diz-se que um programa paralelo que obtém um *speedup* maior que *p* (o número de processos ou *threads*) tem *speedup* superlinear. No entanto, muitos autores não consideram os programas que superam as "limitações de recursos" como tendo aceleração superlinear. Por exemplo, um programa que deve usar armazenamento secundário para seus dados quando é executado em um sistema de processador único pode ser capaz de acomodar todos os seus dados na memória principal quando executado em um grande sistema de memória distribuída. Dê outro exemplo de como um programa pode superar uma limitação de recursos e obter *speedups* maiores que *p*.

Um exemplo relevante de aceleração superlinear ocorre quando o volume de dados processado por um único processador é suficientemente grande para exceder a capacidade de sua memória cache. Nesse cenário, a execução sequencial sofre penalizações de desempenho significativas devido à alta taxa de falhas de cache (cache misses), o que resulta em acessos frequentes à memória principal, mais lenta.

Ao paralelizar a aplicação entre p processadores, o volume total de dados é dividido aproximadamente por p, reduzindo a carga de dados atribuída a cada processador. Essa divisão pode permitir que os dados de cada processador se ajustem integralmente à sua memória cache, minimizando as falhas de cache e maximizando o aproveitamento do acesso rápido à memória.

Como consequência, o desempenho individual de cada processador pode ser significativamente superior ao do processador único, possibilitando a obtenção de um speedup maior que p. Esse ganho adicional decorre não apenas da divisão do trabalho, mas também da eliminação de gargalos relacionados ao acesso à memória.

QUESTÃO 16

Sendo n o tamanho do problema e p o número de *threads* (ou processos), suponha $T_{serial} = n$ e $T_{parallel} = n / p + log_2 (p)$, onde os tempos estão em microssegundos. Se aumentarmos p por um fator de k, encontre uma fórmula para quanto precisaremos aumentar n para manter a eficiência constante.

(a) Quanto devemos aumentar n se dobrarmos o número de processos de *8* para *16*?

Sabemos que a fórmula da eficiência é:

$$E = \frac{T_{Serial}}{p * T_{Parallel}}$$

Substituindo pelas equações fornecidas, temos:

$$E = \frac{n}{p*(\frac{n}{p}+log_2(p))}$$

Para manter a eficiência constante e manter o fator de aumento n de *8* para *16* devemos igualar as duas formulas de eficiência e multiplicar o x pelo n, para descobrirmos o valor de x, logo:

$$\frac{n}{8*(\frac{n}{8}+log_2(8))} = \frac{xn}{16*(\frac{xn}{16}+log_2(16))}$$

$$\frac{n}{n+8log_2(8)} = \frac{xn}{xn+16log_2(16)}$$

$$\frac{n}{n+24} = \frac{xn}{xn+64}$$

$$n(xn + 64) = xn(n + 24)$$

$$xn + 64 = \frac{xn(n+24)}{n}$$

$$xn + 64 = xn + 24x$$

$$24x = xn + 64 - xn$$

$$x = \frac{64}{24}$$

$$x = \frac{8}{3} \text{ ou } 2.67$$

Logo, quando dobrarmos o número de processos precisamos multiplicar o n por 2.67 para manter a eficiência constante.

(b) O programa paralelo é escalável?

Não, porque se aumentarmos a quantidade de processos, é necessário aumentar a quantidade de problemas, para o programa manter a eficiência, então não é escalável.

QUESTÃO 17

Um programa que obtém *speedup* linear é fortemente escalável? Explique sua resposta.

Sim. Pois conforme o slide do capítulo 2, para um programa de *speedup* linear temos que:

$$S = P$$

Logo, como a Eficiência é dado por:

$$E = \frac{S}{P}$$

Vamos ter que:

$$E = \frac{P}{P} = 1$$

Então, quando aumentamos *p* e mantemos o tamanho do problema, caracteriza um programa fortemente escalável com $E = 1$.

QUESTÃO 18

Bob tem um programa que deseja cronometrar com dois conjuntos de dados, *input_data1* e *input_data2*. Para ter uma ideia do que esperar antes de adicionar funções de temporização ao código de seu interesse, ele executa o programa com os dois conjuntos de dados e o comando shell Unix *time*:

```
$ time ./bobs_prog < input_data1
real 0m0.001 s
user 0m0.001 s
sys 0m0.000 s
$ time ./bobs_prog < input_data2
real 1m1.234 s
user 1m0.001 s
sys 0m0.111 s
```

A função de temporização que Bob está usando tem resolução de milissegundos.

(a) Bob deveria usá-la para cronometrar seu programa com o primeiro conjunto de dados? Por que?

Bob não deverá utilizar essa função de temporização, pois o tempo de execução com o *input_data1* foi de 1 milissegundo, *real 0m0.001s*, o que está no limite da resolução da função de temporização que está sendo utilizada. Como a resolução é de milissegundos, esse valor pode ser impreciso ou até sujeito a flutuações causadas por ruído do sistema operacional.

(b) E quanto ao segundo conjunto de dados? Por que?

Bob pode utilizar a função de temporização com o segundo conjunto de dados, pois os resultados são estáveis, o tempo de execução foi de mais de um minuto *(real 1m1.234s)*, o que está muito acima da resolução mínima da função de milissegundos.

QUESTÃO 19

Escreva um pseudocódigo para a soma global estruturada em árvore para somar vetores.

(a) Primeiro considere como isso pode ser feito em um ambiente de memória compartilhada.

- Quais variáveis são compartilhadas e quais são privadas?

```
C/C++
//Compartilhadas
p = quantidade de núcleos;
tamanho_vetor = tamnho do vetor;
vetores[p][tamanho_vetor];

//Privadas
my_rank = índice do núcleo;
rank_parceiro = índice da dupla;
expoente = 2;
distancia = 1;
meu_vetor[] = vetor de cada núcleo;

while (distancia < p){
	if(my_rank % distancia == 0){
		if(my_rank % expoente == 0){
			rank_parceiro = my_rank + distancia;

			for (j = 0; j < tamanho_vetor; j++){
			//Recebo o vetor da dupla e somo elemento por elemento
			meu_vetor[j] += vetores[rank_parceiro][j]
			}

		}else{
			rank_parceiro = my_rank - distancia;
			//Envio meu vetor para a matriz compartilhada
			vetores[my_rank] = meu_vetor;
			break;
		}
	}
	expoente *= 2;
	distancia *= 2;
}
```

(b) Depois considere como isso pode ser feito em um ambiente de memória distribuída.

```
C/C++
p = quantidade de núcleos;
tamanho_vetor = tamnho do vetor;
my_rank = índice do núcleo;
```

```
rank_parceiro = índice da dupla;
expoente = 2;
distancia = 1;
meu_vetor[] = vetor de cada núcleo;
vetor_temp[] = vetor temporario da dupla

while (distancia < p){
	if(my_rank % distancia == 0){
		if(my_rank % expoente == 0){
			rank_parceiro = my_rank + distancia;
			//Recebo o vetor da dupla
			Receber(vetor_temp[tamanho_vetor], de rank_parceiro);

			for (j = 0; j < tamanho_vetor; j++){
			//somo elemento por elemento
			meu_vetor[j] += vetor_temp[j]
			}

		}else{
			rank_parceiro = my_rank - distancia;
			//Envio meu vetor para a dupla
			Enviar(meu_vetor[], para rank_parceiro);
			break;
		}
	}
	expoente *= 2;
	distancia *= 2;
}
```